APUNTES. LOS ARRAYS O MATRICES (C# / .NET)

Estructuras de datos básicas — Colecciones de tamaño fijo

1. ¿Qué es un array?

Un array (Matriz) es una estructura de datos que almacena varios valores del mismo tipo en una única variable. Se accede a cada elemento mediante un índice (posición), empezando en 0.

2. ¿Por qué usar arrays?

- Para agrupar datos relacionados: notas, precios, edades, etc.
- Para recorrer datos fácilmente con bucles (for/foreach).
- Para tener acceso rápido por índice (lectura/escritura directa).
- Son muy eficientes y están integrados en el lenguaje.

3. Declaración y creación

Un array se declara indicando el tipo y los corchetes []. Luego se crea con new:

```
int[] numeros;            // declaración (aún no creado)
numeros = new int[5];       // creación con 5 posiciones (0..4)

string[] nombres = new string[3]; // array de 3 strings
```

Cuando creas un array con un tamaño, ese tamaño queda fijo. No puedes “añadir” posiciones: si necesitas crecer, normalmente usarás List<T> (lo veremos más adelante).

4. Inicialización rápida (valores en línea)

```
int[] a = new int[] { 10, 20, 30 };
int[] b = { 1, 2, 3, 4 };          // forma abreviada

string[] dias = { "Lun", "Mar", "Mié" };
```

5. Índices: leer y escribir

El índice empieza en 0. El último índice es Length - 1.

```
int[] nums = { 5, 10, 15 };

Console.WriteLine(nums[0]);  // 5
nums[1] = 99;                // cambia el segundo elemento
Console.WriteLine(nums[1]);  // 99
```

Si intentas acceder a un índice fuera de rango (por ejemplo nums[3] en un array de 3 elementos), se lanzará una excepción: IndexOutOfRangeException.

6. Propiedad Length y límites

```
int[] nums = new int[5];
Console.WriteLine(nums.Length);    // 5

// Recorrer con for usando Length
for (int i = 0; i < nums.Length; i++)
{
    Console.WriteLine($"i={i} valor={nums[i]}");
}
```

7. Valores por defecto

Cuando creas un array con new, C# asigna valores por defecto a cada posición:

- int, double, etc. → 0 / 0.0
- bool → false
- char → '\0'
- string y otros tipos referencia → null

```
int[] x = new int[3];        // {0,0,0}
bool[] b = new bool[2];       // {false,false}
string[] s = new string[2];   // {null,null}
```

8. Recorrer arrays: for y foreach

Dos formas típicas:

```
int[] nums = { 2, 4, 6 };

// for: útil si necesitas el índice
for (int i = 0; i < nums.Length; i++)
{
    Console.WriteLine($"Posición {i}: {nums[i]}");
}

// foreach: más limpio si solo quieres los valores
foreach (int n in nums)
{
    Console.WriteLine(n);
}
```

9. Arrays y entrada de usuario

Ejemplo: pedir 5 notas y calcular la media.

```
double[] notas = new double[5];
double suma = 0;

for (int i = 0; i < notas.Length; i++)
{
    Console.Write($"Nota {i+1}: ");
    notas[i] = double.Parse(Console.ReadLine());
    suma += notas[i];
}

double media = suma / notas.Length;
Console.WriteLine($"Media: {media:0.00}");
```

10. Métodos útiles de la clase Array

En C#, los arrays tienen soporte a través de la clase estática Array. Algunos métodos comunes:

- Array.Sort(arr) → ordena el array.
- Array.Reverse(arr) → invierte el orden.
- Array.IndexOf(arr, valor) → devuelve el índice de la primera coincidencia o -1.

- Array.Copy(origen, destino, cantidad) → copia elementos.

Ejemplo rápido:

```
int[] nums = { 5, 2, 9, 1 };
Array.Sort(nums);      // {1,2,5,9}
Array.Reverse(nums);   // {9,5,2,1}

int pos = Array.IndexOf(nums, 5); // 1
```

11. Copias: referencia vs copia real

Ojo: si haces esto, NO estás copiando el contenido, solo la referencia:

```
int[] a = { 1, 2, 3 };
int[] b = a;           // b apunta al MISMO array que a
b[0] = 99;
Console.WriteLine(a[0]); // 99 (¡se ha cambiado también!)
```

Para copiar el contenido, usa Copy o Clone:

```
int[] a = { 1, 2, 3 };

int[] b = (int[])a.Clone();      // copia real
b[0] = 99;

Console.WriteLine(a[0]); // 1
Console.WriteLine(b[0]); // 99
```

12. Arrays multidimensionales (2D)

Un array 2D se parece a una tabla (filas y columnas). Se declara con comas dentro de los corchetes.

```
int[,] tabla = new int[2, 3]; // 2 filas, 3 columnas

tabla[0, 0] = 10;
tabla[0, 1] = 20;
tabla[1, 2] = 99;

Console.WriteLine(tabla[1, 2]); // 99
```

Recorrer 2D con bucles:

```
for (int f = 0; f < tabla.GetLength(0); f++)       // filas
{
    for (int c = 0; c < tabla.GetLength(1); c++)   // columnas
    {
        Console.Write(tabla[f, c] + " ");
    }
    Console.WriteLine();
}
```

13. Jagged arrays (array de arrays)

Un jagged array es un 'array de arrays'. Cada fila puede tener una longitud diferente.

```
int[][] jagged = new int[3][];

jagged[0] = new int[] { 1, 2 };
jagged[1] = new int[] { 10, 20, 30 };
jagged[2] = new int[] { 7 };

Console.WriteLine(jagged[1][2]); // 30
```

14. Errores típicos (y cómo evitarlos)

- Usar índices fuera de rango → revisa i < arr.Length.
- Olvidar inicializar (arr = new ...) → la variable queda null y da NullReferenceException.
- Confundir copia con referencia → usa Clone/Copy si quieres duplicar.
- No validar entradas del usuario → usa TryParse cuando sea posible.

15. Resumen rápido

- Array = colección de elementos del mismo tipo, tamaño fijo.
- Índices: 0..Length-1.
- Recorrido: for (índices) y foreach (valores).
- Métodos útiles: Sort, Reverse, IndexOf, Copy.
- Existen arrays 2D y jagged arrays (array de arrays).